Звіт з лабораторної роботи № 2

Нерекурсивний Фібоначчі. Переведення в двійкову та шістнадцяткову системи числення

1. Нерекурсивний варіант функції Фібоначчі

У завданні наведена рекурсивна функція: F(n) = -1 при n < 1, F(1) = F(2) = 1, інакше F(n) = F(n-1) + F(n-2). Вона робить O(2^n) викликів. Ітеративна версія тримає два попередні числа в змінних і за один прохід циклу отримує наступне: O(n) часу та O(1) пам'яті.

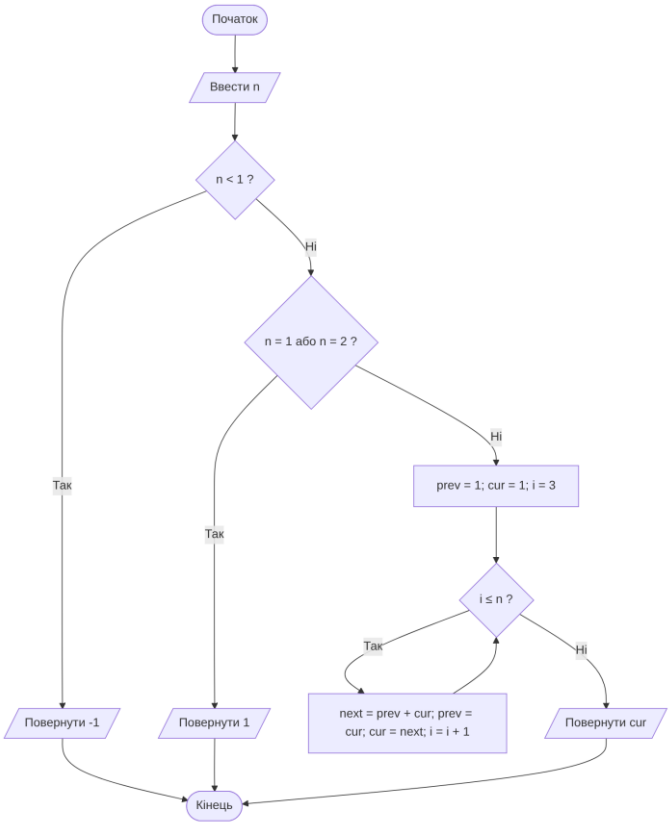


Рис. 1. Блок-схема ітеративного обчислення числа Фібоначчі (Mermaid)

```
function fibonacci(n) {
  if (n < 1) return -1;
  if (n === 1 || n === 2) return 1;
  let prev = 1, cur = 1;
  for (let i = 3; i <= n; i++) {
    const next = prev + cur;
    prev = cur;
    cur = next;
  }
  return cur;
}
```

2. Переведення в двійкову та шістнадцяткову системи

Число ділять на основу, залишки записують у зворотному порядку. Одна функція toBase працює для основ 2..16, а toBinary і toHex - її окремі випадки. Вбудований toString(base) не використовується в алгоритмі, лише для перевірки результату. Приклад для 255: двійкова 11111111 (8 кроків ділення на 2), шістнадцяткова FF (255 mod 16 = 15, 15 mod 16 = 15).

```
const DIGITS = "0123456789ABCDEF";
function toBase(x, base) {
  if (!Number.isInteger(x) || x < 0) throw new Error("Потрібне невід'ємне ціле");
  if (x === 0) return "0";
  let result = "";
  while (x > 0) {
    result = DIGITS[x % base] + result;
    x = Math.floor(x / base);
  }
  return result;
}
const toBinary = (x) => toBase(x, 2);
const toHex = (x) => toBase(x, 16);
```

3. Результат виконання

```
=== Фібоначчі (ітеративно) ===
1 1 2 3 5 8 13 21 34 55 89 144 233 377 610
F(50) = 12586269025

=== Системи числення ===
0 -> bin 0 | hex 0 | перевірка: true
5 -> bin 101 | hex 5 | перевірка: true
10 -> bin 1010 | hex A | перевірка: true
255 -> bin 11111111 | hex FF | перевірка: true
1024 -> bin 10000000000 | hex 400 | перевірка: true
48879 -> bin 1011111011101111 | hex BEEF | перевірка: true
```

F(50) = 12586269025 збігається з відомим значенням. Для всіх тестових чисел toBinary/toHex збігаються з вбудованим toString (перевірка = true). Повний файл: lab2.js.

4. Таблиця відповідей на контрольні запитання

Номер питання	Відповідь
1	4) Рекурсивну функцію для n!. Запис F(0) = 1, F(n) = n·F(n-1) при n > 0 - рекурсивне означення факторіала.
2	7) O(2^n). Кількість викликів c(n) = 1 + c(n-1) + c(n-2), c(0) = c(1) = 1: 1, 1, 3, 5, 9, 15, 25, тобто для f(6) дерево має 25 вузлів; зростання приблизно в 1,618 разу на крок обмежується згори O(2^n).
3	Немає умови завершення (бази рекурсії): функція завжди викликає саму себе, рекурсія нескінченна й закінчується переповненням стека (stack overflow).
4	Умова numb <= 1 є, але виклик fac(numb+1) збільшує аргумент, тобто віддаляє від бази. Для numb > 1 рекурсія нескінченна. Правильно: fac(numb-1).
5	Ймовірно: 1) лінійна, 4) хвостова, 5) обопільна (взаємна), 6) двійкова, 7) вкладена. Умовна, циклічна й стекова не є типами рекурсії: умова - перевірка бази, цикл - ітерація, стек - механізм викликів. Перелік у слайді неоднозначний (вкладену можна не зараховувати), тож уточніть за конспектом лекції.
6	1) Лінійна: один рекурсивний виклик Factorial(n-1); після нього ще множення на n, тому не хвостова.
7	4) Хвостова: рекурсивний виклик - остання дія, проміжний результат накопичується в параметрі a.
8	6) Двійкова: у тілі два рекурсивні виклики - Fib(no-1) і Fib(no-2).
9	5) Обопільна (взаємна): isEven викликає isOdd, а isOdd викликає isEven.